INSIDE STLC ACADEMY

QA LEADERSHIP ACADEMY

Product Risk Matrix

Rank what could go wrong by probability and impact, then aim your testing effort where it earns its keep.

Purpose

A product risk matrix turns "we should test everything" into a defensible plan. You score each risk on two axes — how likely it is to occur (probability) and how badly it hurts if it does (impact) — and let the combined score decide how much test effort it deserves. It is the single most useful artefact for justifying where your team spends its time, and for saying no to low-value work without sounding defensive.

The matrix is a communication tool as much as a planning tool. A VP Engineering does not want a list of 400 test cases; she wants to know which handful of risks could genuinely hurt the business, and that you are covering them.

When to use it

- At the start of a release or programme, to shape the test approach before writing a single case.
- When you are time-boxed and cannot test everything — the matrix decides what gets cut safely.
- When you need to justify test effort (or extra time) to product and engineering leaders.
- During a go/no-go conversation, to show residual risk in plain terms.
- Whenever a new area of the product changes and you must re-scope coverage.

Instructions

1. Identify risks as concrete "what could go wrong" statements, not vague areas. "Payment is captured but no order is created" is a risk; "payments" is not.
2. Score probability on a 1-5 scale: 1 Rare, 2 Unlikely, 3 Possible, 4 Likely, 5 Almost certain. Base it on evidence — change size, code age, past defects, complexity, team familiarity.
3. Score impact on a 1-5 scale: 1 Negligible, 2 Minor, 3 Moderate, 4 Major, 5 Severe. Weigh customer harm, revenue, data integrity, legal/compliance and reputation.
4. Multiply the two scores to get a risk rating from 1 to 25, and band it: 1-4 Low, 5-9 Medium, 10-15 High, 16-25 Critical.
5. Map the band to a test-effort response so the plan is automatic and consistent across the team.
6. Review the matrix with product and engineering, and record who agreed. Shared ownership of risk is the whole point.

The probability × impact grid

Probability \ Impact	Negligible (1)	Minor (2)	Moderate (3)	Major (4)	Severe (5)
Almost certain (5)	5 · Med	10 · High	15 · High	20 · Critical	25 · Critical
Likely (4)	4 · Low	8 · Med	12 · High	16 · Critical	20 · Critical
Possible (3)	3 · Low	6 · Med	9 · Med	12 · High	15 · High
Unlikely (2)	2 · Low	4 · Low	6 · Med	8 · Med	10 · High
Rare (1)	1 · Low	2 · Low	3 · Low	4 · Low	5 · Med

Band	Rating	Test-effort response
Critical	16-25	Deep, planned testing. Multiple techniques, negative and boundary cases, dedicated review, automated regression. Blocks release if unresolved.
High	10-15	Thorough scripted and exploratory testing of the risk. Regression coverage. Explicit sign-off on residual risk.
Medium	5-9	Targeted testing, mostly happy path plus the obvious failure modes. Time-box it.
Low	1-4	Light touch or sanity check only. Acceptable to defer or skip under time pressure — record the decision.

Worked example

Northstar Digital is releasing a change to the Payments squad's checkout flow that also touches the legacy in-house billing service. The QA lead scores five candidate risks. Probability reasoning draws on the known context: the billing monolith is old and fragile, shared staging is unstable, and payments span a third-party provider plus legacy code.

Risk (what could go wrong)	Prob.	Impact	Rating	Band	Why scored this way
Payment is captured by the provider but no order is created (customer charged, nothing delivered)	3 Possible	5 Severe	15	High	Split responsibility between provider and legacy billing makes a mismatch plausible; customer harm and refunds are severe.
Billing data corruption — a wrong amount written to the legacy billing records	2 Unlikely	5 Severe	10	High	The change is well isolated, but corrupted financial records are catastrophic and hard to unwind, so it still lands High.
Intermittent checkout failure under load or on flaky staging	3 Possible	4 Major	12	High	Shared staging is unstable and the symptom is intermittent; lost sales and lost trust make the impact Major.
Analytics event for "purchase" does not fire, so the funnel report is wrong	3 Possible	2 Minor	6	Medium	Reasonably likely with a checkout change, but it degrades a report rather than harming a customer.
Minor visual defect — the discount label is slightly misaligned on mobile	4 Likely	1 Negligible	4	Low	Cosmetic churn is common with UI changes, but the consequence is trivial.

What the plan then becomes: the two High payment/billing risks and the intermittent-failure risk absorb the bulk of the effort — deep exploratory testing of the capture-to-order path, deliberate negative and reconciliation checks against billing, and a repeated run to expose intermittence. The analytics gap gets a single targeted check. The visual defect is logged for later and would not block the release. The QA lead can now stand in front of Tom Fielding, the Head of Product, and say precisely where the time is going and why the cosmetic issue is not getting any.

Blank reusable version

Risk (what could go wrong)	Prob. (1-5)	Impact (1-5)	Rating	Band	Test response / owner

Reuse the probability, impact and band definitions above unchanged so scores stay comparable across releases and squads.

Common mistakes

COMMON MISTAKES

Scoring everything High so nothing is prioritised; treating the numbers as science rather than a shared conversation; scoring probability on gut feel with no reference to change size or defect history; forgetting that a rare event with severe impact (data corruption) still deserves real effort; and building the matrix alone, so product and engineering never own the risk with you.

- Vague risks ("the API") that cannot be tested for — always phrase risks as a specific failure.
- Never revisiting the matrix, so it reflects the plan you imagined, not the product you shipped.
- Confusing severity of a defect with impact of a risk — impact is about business consequence, not how loud the crash is.

How to interpret the results

- Critical and High risks are your test plan. If you can only test some things, test these — the matrix is your written justification for the rest.
- A cluster of Critical risks in one area is a signal to raise scope or timeline before the release, not after.
- Low risks are candidates to defer, automate later, or accept explicitly. "We chose not to test X because it scored 3" is a professional statement; silence is not.
- A matrix where nearly everything is High usually means the scoring is uncalibrated — recheck probability against real evidence.
- Residual risk is what remains High after testing. That, not a pass rate, is what you report at go/no-go.

INSIDE STLC PRO TIP

Score probability from evidence, not instinct. Before you write a number, ask: how large is the change, how old and fragile is the code, and how many defects has this area produced before? A matrix built on evidence survives challenge in the room; one built on feelings collapses the moment an engineer disagrees.